

SIMATS ENGINEERING
ASSESSMENT TOOL 4 - HIGH
(ANALYTICAL PROBLEM SOLVING QUESTIONS)

COURSE CODE: ITA04

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

QUESTIONS: 10

Total Marks: 100

ANNEXURE A

ANALYTICAL PROBLEM SOLVING QUESTIONS

Code of Conduct:

I [Aryan.ks/](https://aryan.ks/) 192325051 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink, appearing to read 'Aryan', is written on a light gray rectangular background.

Analytical Problem Solving Questions

Question 1: You are running code to replace extreme anomalies in a dataset using a standard if-else block instead of a vectorized tool.

```
R
set.seed(42)
sensor_readings <- c(102, 95, 110, 88)
if (sensor_readings > 100) {
  sensor_readings <- 100
} else {
  sensor_readings <- sensor_readings
}
```

What will happen when this code runs in R?

ANS:

Error: the condition has length > 1 and only the first element will be used. Since the first value (102) is >100, **sensor_readings** becomes 100

Question 2: Evaluate the following conditional statement processing a vector variable val.

```
R
val <- c(6, 12, 18)

result <- if (length(val) == 3 && val > 10) {
  "Condition Met"
} else {
  "Condition Failed"
}
```

What is the behavior and output of this code snippet?

Error: && expects a single logical value but **val > 10 is a vector. Only the first element is used (**6>10** is FALSE), so "Condition Failed" is returned (with a warning)**

Question 3: Consider this operation involving logical conditions and vector comparisons:

```
R
x <- c(TRUE, FALSE, TRUE)
y <- c(FALSE, FALSE, TRUE)
```

```
output_1 <- x & y
output_2 <- x && y
```

What are the exact contents of output_1 and output_2?

output_1 = c(FALSE, FALSE, TRUE) and output_2 = FALSE

Question 4: Examine this custom scoring mathematical utility function:

R

```
calculate_bonus <- function(score) {  
  if (score > 90) {  
    bonus <- score * 0.1  
  } else {  
    bonus <- 0  
  }  
}  
final_payout <- calculate_bonus(95)
```

What is the exact value stored inside the variable final_payout?

final_payout = 9.5 (the last evaluated expression is **bonus**)

Question 5: A data scientist loops through an index array to select rows based on arithmetic indexes:

R

```
index_vector <- c(10, 15, 22)  
calculated_values <- (index_vector %% 4) + (index_vector %/% 4)
```

What vector sequence is generated inside calculated_values?

ANS: calculated_values = c(4, 6, 7)

Question 6: Study how the parameters in this function interact with each other:

R

```
compute_metric <- function(base_val, modifier = base_val * 2, scale = modifier / 2) {  
  base_val + modifier + scale  
}  
result_metric <- compute_metric(base_val = 5)
```

What value does result_metric evaluate to?

ANS: result_metric = 20

Question 7: You pass an environmental monitoring dataset into a data processing clean-up function:

R

```
air_quality <- list(city = "Mumbai", readings = c(45, 62, 58))  
update_readings <- function(dataset) {  
  dataset$readings <- 90  
  dataset$status <- "Updated"  
  return(dataset$readings)  
}  
extracted_readings <- update_readings(air_quality)
```

After execution, what are the values of air_quality\$readings and air_quality\$status?

`extracted_readings = 90`; `air_quality$readings` remains `c(45, 62, 58)` and `air_quality$status` remains `NULL` because lists are passed by value (copy-on-write)

Question 8: You pass a tracking mathematical processing function directly into another system operation engine:

R

```
math_factory <- function(type) {  
  if (type == "double") {  
    function(n) n * 2  
  } else {  
    function(n) n * 3  
  }  
}
```

```
selected_operation <- math_factory("double")
```

```
final_output <- selected_operation(selected_operation(4))
```

What is the final numerical value evaluated inside final_output?

ANS: final_output = 16

Question 9: Trace the conditional flags inside this infinite loop construct:

R

```
state_val <- 1
```

```
accumulation <- 0
```

```
repeat {
```

```
  state_val <- state_val * 2
```

```
  if (state_val > 5) {
```

```
    state_val <- state_val - 3
```

```
  }
```

```
  accumulation <- accumulation + state_val
```

```
  if (accumulation >= 10) {
```

```
    break
```

```
  }
```

```
}
```

What is the value of accumulation when the loop successfully breaks and finishes?

accumulation = 11

Question 10

Analyze this recursive tracking element filter function that parses through numbers:

R

```
process_stream <- function(vec) {
```

```
  if (length(vec) == 0) {
```

```
    return(0) }
```

```
  head_element <- vec
```

```
tail_elements <- vec[-1]
modifier <- if (head_element %% 3 == 0) 10 else 2
return(modifier + process_stream(tail_elements)) }
stream_output <- process_stream(c(3, 5, 9))
What is the computed total returned in stream_output?
```

stream_output = 22

Question 11. Analyze the following R function intended to clean data by replacing negative values with zero, or returning NULL if the vector contains too many missing values:

```
R
clean_sensor_data <- function(readings, max_na = 2) {
  if (sum(is.na(readings)) > max_na) {
    return(NULL)
  }
  ifelse(readings < 0, 0, readings)
}
```

If you run `clean_sensor_data(c(-1, 5, NA, 10, NA))`, what unexpected behavior or error will occur, and why?

- A) It throws a syntax error because `return(NULL)` is incompatible with the vectorised output of `ifelse()`.
- B) It returns `NULL` because the number of missing values matches `max_na` exactly.
- C) It returns `c(NA, 5, NA, 10, NA)` because `ifelse()` evaluates `readings < 0` as `NA` for the missing values, preserving them.
- D) It fails with missing value where `TRUE/FALSE` needed because `sum(is.na(readings))` evaluates to `NA`.

Question 12. Consider a scenario where an engineer attempts to implement an option where an evaluation threshold can either be explicitly set or automatically determined based on the median of the data. They write:

```
R
evaluate_performance <- function(scores, threshold = median(scores)) {
  scores[scores > threshold]
}
```

If a user calls this function using `evaluate_performance(scores = c(80, 90, 70, 60, 100), threshold = threshold_val)`, where `threshold_val` is an undefined variable in both the local and global environments, what will happen?

- A) R throws an error immediately upon calling the function because `threshold_val` cannot be found.**
- B) The function executes successfully and sets `threshold` to the median (80) because default arguments handle missing variables.
- C) The function enters the body successfully but throws an error the moment `threshold` is evaluated in the indexing expression.
- D) R searches the parent environment, dynamically generates a `threshold_val` set to 0, and returns all scores.

Question 13. Examine this optimization loop meant to find the first index where a cumulative sum exceeds 100:

R

```
find_target_index <- function(vec) {  
  total <- 0  
  for (i in 1:length(vec)) {  
    total <- total + vec[i]  
    if (total > 100) {  
      return(i)  
    }  
  }  
  return(-1)  
}
```

If this function is evaluated with an empty numeric vector `v <- numeric(0)`, what will it return, and what structural flaw does this expose?

- A) It returns -1 because the loop condition is skipped safely when the vector is empty.
- B) It throws an out-of-bounds memory pointer exception because `numeric(0)` lacks index 1.
- C) It executes the loop twice because `1:length(vec)` evaluates to the sequence `1:0`, causing a runtime error when accessing `vec`.**
- D) It returns NULL because for loops implicitly overwrite early explicit `return()` expressions.

Question 14. A developer needs to modify a heavy configuration list in place inside a loop to minimize overhead. They write:

R

```
update_config <- function(conf) {  
  conf$updated <- TRUE  
  conf$timestamp <- Sys.time()  
  return(conf)  
}  
system_config <- list(version = "1.0", updated = FALSE)  
for (i in 1:1000) {  
  system_config <- update_config(system_config)  
}
```

Based on R's memory management rules (No Pointers / Copy-on-Write), what is the architectural consequence of this code?

- A) Memory usage remains perfectly flat because R treats the list as a reference pointer inside the loop.
- B) R creates a modified local copy of the list during each iteration, incurring allocation overhead despite appearing to update "in place."**
- C) The code crashes on the second iteration because global objects are locked against modifications originating within functions.
- D) The `system_config` variable outside the loop retains `updated = FALSE` because functions cannot pass modified lists back.

Question 15. Review the following compound logical statement designed to validate a single experimental run payload:

R

```
validate_run <- function(status_code, metrics) {  
  if (!is.null(status_code) && status_code == 200 & max(metrics) < 100) {  
    return("Pass")  
  } else {  
    return("Fail")  
  }  
}
```

If this function is executed with `status_code = NULL` and `metrics = c(10, 50, 80)`, what is the outcome?

A) It safely returns "Fail" because the short-circuiting operator `&&` stops evaluation when it sees `status_code` is `NULL`.

B) It throws an error because the single `&` operator forces the evaluation of `status_code == 200`, which outputs a zero-length logical vector.

C) It returns "Pass" because `!is.null(status_code)` evaluates to `FALSE`, which sets the entire chain to true under lazy validation rules.

D) It crashes because `max(metrics)` cannot be evaluated side-by-side with null objects.

Question 16. A script computes a deep mathematical expansion using a recursive strategy:

R

```
compute_depth <- function(val, accumulator = 0) {  
  if (val <= 0) return(accumulator)  
  compute_depth(val - 1, accumulator + log(val))  
}
```

When testing with large values (e.g., `compute_depth(25000)`), the program halts with a maximum nesting depth exceeded error. Which structural change provides the most robust solution in standard R?

A) Enclose the recursive call inside a `tryCatch()` environment block to automatically increase stack limits.

B) Convert the recursive algorithm into an iterative structure using a vectorised calculation or a standard while loop.

C) Convert `val` to a pointer reference using an explicit assignment expression to bypass scope limits.

D) Change the explicit `return(accumulator)` into an implicit assignment statement to save stack frames.

Question 17. An analyst builds a functional dispatch pipeline where mathematical operations are stored in a named list and called dynamically:

R

```
pipeline <- list(  
  double = function(x) x * 2,  
  square = function(x) x ^ 2  
)  
run_pipeline <- function(vec, step) {
```

```
pipeline$step(vec)
}
```

What happens when running `run_pipeline(c(2, 4), "square")`?

- A) It returns `c(4, 16)` by matching the string "square" to the named element inside the list.
- B) It throws an error because lists can only store atomic values, not functional objects.
- C) It fails with an error because `pipeline$step` looks for a literal element named "step" inside the list instead of evaluating the variable's value.**
- D) It returns `NULL` because functions passed into pipelines automatically lose their internal scopes.

Question 18. Analyze the execution flow of the following nested loop structure:

R

```
matrix_search <- function() {
  for (row in 1:3) {
    for (col in 1:3) {
      if (row == 2 && col == 2) {
        break
      }
      print(paste(row, col, sep = "-"))
    }
  }
}
```

What is the final coordinate pair printed to the console before the function finishes executing?

- A) "2-1"
- B) "3-3"**
- C) "2-2"
- D) "1-3"

Question 19. Evaluate the following conditional validation check handling a vector of database transaction amounts:

R

```
verify_transactions <- function(amounts) {
  if (amounts > 0) {
    "All Positive"
  } else {
    "Contains Zero or Negative"
  }
}
```

If called with `verify_transactions(c(100, -50, 200))`, what does this function return, and why?

- A) It returns "Contains Zero or Negative" because it calculates an implicit `all()` operation across the array.
- B) It throws an execution error because conditional `if` blocks cannot accept numeric vector variables.
- C) It returns "All Positive" along with a warning message because it only checks the first element (`100 > 0`) and ignores the rest.**

D) It returns c("All Positive", "Contains Zero or Negative", "All Positive") through implicit vector distribution.

Question 20. Consider the behavior of missing values in logical comparisons. A script filters a vector using an index derived from an operator expression:

R

```
data_points <- c(12, 15, NA, 18)
```

```
filtered <- data_points[data_points > 14]
```

What is the exact structure and content of the filtered variable after running this expression?

A) c(15, 18)

B) c(15, NA, 18)

C) c(12, 15, 18)

D) It throws a missing value where TRUE/FALSE needed termination error.

RUBRICS FOR CALCULATION

Analytical Problem Solving					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding ; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretatio n with minor omissions	Limited or unclear interpretation	No interpretatio n
------------------------------	-----	--	--	---	--------------------------